mechanism instead of the components communicating with each other directly. An event will eventually get to the subscribers: in this case, to the `Mentions` component. Once again, you can model the solution to this problem by sending and receiving messages.

1.5.3 Scaling with Akka and failure: Asynchronous decoupling

Users should be able to continue to have conversations even if the `Mentions` component crashes. The same goes for the `TeamFinder` component: existing conversations should be able to continue. Conversations can continue to publish events while subscribers—like the `Mentions` component and the `TeamFinder` object—crash and restart. The `NotifyUser` component can keep track of connected web browsers and send `UserMentioned` messages directly to the browser when they occur, relieving the application from polling.

This event-driven approach has a few advantages:

- It minimizes direct dependencies between components. The `Conversation` actor doesn't know about the Mentions object and could not care less about what happens with the event. The `Conversation` can continue to operate when the `Mentions` object crashes.
- The components of the application are loosely coupled in time. It doesn't matter if the `Mentions` object gets the events a little later as long as it gets them eventually.